

1. Introduction

The base project requirements are implemented (Trending, Search, Movie Details with cast/crew, Person Details, and persistent Favourites). Beyond these requirements, this project adds several innovation-focused features to improve performance, user experience, and interactivity using IndexedDB persistence and additional TMDB querying logic.

2. Summary of Innovations (Extra Functionality)

Innovation	What it does (User-facing)	Technical implementation (high level)
Caching with TTL (12 hours)	Faster page loads and reduced repeated API calls for frequently requested data	Implemented in <code>src/app/services/tmdb.ts</code> using <code>src/app/services/app-db.ts</code> TTL cache
Search History + Dropdown UI	Saves the last search queries and shows them in a dropdown for quick reuse	AppDb stores queries in IndexedDB; Home loads and displays history; Home dropdown + clear option
Autocomplete Suggestions (2+ chars, debounced)	Predictive suggestions while typing, merged into the same dropdown	<code>TmdbService.searchMoviesAutocomplete()</code> with caching; Home debounces input before fetching suggestions
Recently Viewed (Movies only, Home while Trending)	Tracks movies opened from Movie Details and displays them on Home when trending is active	Movie Details saves “recently viewed” after successful TMDB overview load; Home shows it only in trending mode
Favourites Insights Dashboard	Improves favourites management with filter, sorting, count, and “clear all favourites”	<code>src/app/favourites/favourites.page.ts/html</code> sorts + filters the persisted favourites and provides management actions

3. Detailed Innovation Descriptions

3.1. Innovation 1: Caching with TTL (12 hours)

To improve responsiveness and reduce the load on TMDB, the application caches key API responses for 12 hours.

User benefit

- Repeated visits to Home/Trending, search, movie credits, and person details feel faster.
- Offline-like resilience is improved because cached results may still be available within the TTL window.

Implementation details

- `src/app/services/app-db.ts` provides a generic cache store `tmdbCache` with:
 - `expiresAt` timestamps
 - `getCached()` and `setCached()` methods
- `src/app/services/tmdb.ts` uses unique cache keys such as:
 - `trending:day`
 - `search:<normalized query>`
 - `movieCredits:<movieId>`
 - `movieOverview:<movieId>`
 - `person:<personId>`
- When a cached value exists and has not expired, the service returns it immediately; otherwise it fetches from TMDB and stores the result.

3.2. Innovation 2: Search History Dropdown + Clear

This feature saves the user's search strings and displays them in a dropdown to reduce repetitive typing.

User benefit

- Users can quickly revisit previous searches.
- A "Clear search history" control resets the stored queries.

Implementation details

- AppDb maintains an IndexedDB object store searchHistory.
- TmdbService.searchMovies() records the query string after a successful search.
- src/app/home/home.page.ts/html:
 - loads history on ionViewWillEnter()
 - displays it under the search input using a dropdown
 - supports clearing history via clearSearchHistory()

3.3. Innovation 3: Autocomplete Suggestions (2+ chars, debounced) in the same dropdown

Autocomplete improves discovery by showing likely movie matches while the user types.

User benefit

- Suggestions appear after the user enters 2+ characters.
- The dropdown includes both:
 - Recent Searches
 - Autocomplete Suggestions
- Suggestions include the movie title and an overview snippet (shortened text).

Implementation details

- src/app/services/tmdb.ts adds:
 - searchMoviesAutocomplete(query)
- The autocomplete method uses the same caching mechanism (TTL) but intentionally does not write to search history to avoid polluting the history with partial typing.
- src/app/home/home.page.ts debounces user input (e.g., ~450ms) and then fetches suggestions, storing them in suggestions.
- src/app/home/home.page.html renders "Suggestions" inside the same dropdown UI.

3.4. Innovation 4: Recently Viewed Movies (Home-only, while trending is active)

To increase personalization, the application tracks movies the user opens.

User benefit

- On the Home page, a "Recently viewed" section appears so users can return to what they last checked.
- It is shown only during the Trending view (not while the user is viewing search results).
- Movies are saved only after Movie Details successfully loads the TMDB overview (ensuring the stored item is valid and display-ready).

Implementation details

- `src/app/services/app-db.ts` provides:
 - a `recentMovies` store containing `MovieDisplay` + `viewedAt`
- `src/app/movie-details/movie-details.page.ts`:
 - saves recently viewed data after TMDB overview/title/poster are loaded successfully
- `src/app/home/home.page.ts/html`:
 - loads recent movies from IndexedDB
 - displays them only when `showingSearchResults` is false

3.5. Innovation 5: Favourites Insights Dashboard (Filter/Sort/Clear/Count)

This extra functionality enhances the Favourites experience beyond a simple list.

User benefit

- Displays number of favourites.
- Provides filter-by-title input.
- Provides A-Z / Z-A sorting.
- Adds a "Clear all favourites" action with confirmation.

Implementation details

- `src/app/favourites/favourites.page.ts/html`:
 - loads persisted favourites from IndexedDB
 - applies filter + sort on the loaded list
 - clears favourites by removing items from IndexedDB (using the existing favourites persistence layer)

4. Conclusion

These innovations improve both technical quality (performance and persistence through IndexedDB TTL caching) and user experience (autocomplete, search history dropdown, recently viewed, and improved favourites management). Together, they provide a more interactive, faster, and user-friendly mobile movie experience.